

# Rockchip RV1126B Linux SDK 快速入门

---

文档标识: RK-JC-YF-A47

发布版本: V1.1.0

日期: 2025-09-20

文件密级: ☐绝密 ☐秘密 ☐内部资料 ☒公开

## 免责声明

本文档按“现状”提供，瑞芯微电子股份有限公司（“本公司”，下同）不对本文档的任何陈述、信息和内容的准确性、可靠性、完整性、适销性、特定目的性和非侵权性提供任何明示或暗示的声明或保证。本文档仅作为使用指导的参考。

由于产品版本升级或其他原因，本文档将可能在未经任何通知的情况下，不定期进行更新或修改。

## 商标声明

“Rockchip”、“瑞芯微”、“瑞芯”均为本公司的注册商标，归本公司所有。

本文档可能提及的其他所有注册商标或商标，由其各自所有者所有。

## 版权所有© 2025 瑞芯微电子股份有限公司

超越合理使用范畴，非经本公司书面许可，任何单位和个人不得擅自摘抄、复制本文档内容的部分或全部，并不得以任何形式传播。

瑞芯微电子股份有限公司

Rockchip Electronics Co., Ltd.

地址: 福建省福州市铜盘路软件园A区18号

网址: [www.rock-chips.com](http://www.rock-chips.com)

客户服务电话: +86-4007-700-590

客户服务传真: +86-591-83951833

客户服务邮箱: [fae@rock-chips.com](mailto:fae@rock-chips.com)

前言

概述

本文主要描述了RV1126B Linux SDK的基本使用方法，旨在帮助开发者快速了解并使用RV1126B SDK开发包。

读者对象

本文档（本指南）主要适用于以下工程师：

技术支持工程师

软件开发工程师

各芯片系统支持状态

| 芯片名称              | Uboot版本 | Kernel版本 | Buildroot版本 | Yocto版本 | Debian版本 |
|-------------------|---------|----------|-------------|---------|----------|
| RV1126B、RV1126B-P | 2017.9  | 6.1      | 2024.02     | 5.0     | 12       |

修订记录

| 日期         | 版本     | 作者          | 修改说明                                                                                                                                                                                                   |
|------------|--------|-------------|--------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| 2025-03-26 | V0.0.1 | Caesar Wang | Alpha版本                                                                                                                                                                                                |
| 2025-05-15 | V0.1.0 | Caesar Wang | Beta版本                                                                                                                                                                                                 |
| 2025-06-20 | V1.0.0 | Caesar Wang | Release版本                                                                                                                                                                                              |
| 2025-09-20 | V1.1.0 | Caesar Wang | 1. 更新 <a href="#">配置项说明</a><br>2. 增加 <a href="#">AVS说明</a><br>3. 增加 <a href="#">AI-ISP说明</a><br>4. 增加 <a href="#">AI-Remosaic说明</a><br>5. 增加 <a href="#">FEC说明</a><br>6. 增加 <a href="#">如何读取CPU序列号</a> |

# 目录

## Rockchip RV1126B Linux SDK 快速入门

1. SDK预编译镜像
2. 开发环境搭建
  - 2.1 准备开发环境
  - 2.2 安装库和工具集
    - 2.2.1 检查和升级主机的 python 版本
    - 2.2.2 检查和升级主机的 make 版本
    - 2.2.3 检查和升级主机的 lz4 版本
3. Docker环境搭建
4. 软件开发指南
  - 4.1 开发向导
  - 4.2 Buildroot开发指南
  - 4.3 Debian开发指南
  - 4.4 第三方OS移植
  - 4.5 MCU开发指南
  - 4.6 RKNN开发指南
    - 4.6.1 RKNN-TOOLKIT2
    - 4.6.2 RKNN Runtime
    - 4.6.3 RKNN Driver
    - 4.6.4 RKNN LLM
  - 4.7 实时性Linux开发说明
    - 4.7.0.1 PREEMPT\_RT Patch
    - 4.7.0.2 Xenomai Cobalt Mode
  - 4.8 RKIPC开发指南
  - 4.9 AVS 多目拼接
  - 4.10 AI-ISP 图像增强
  - 4.11 AI-Remosaic 像素重排
  - 4.12 FEC 鱼眼畸变校正
  - 4.13 软件更新记录
5. 硬件开发指南
  - 5.1 芯片资料
  - 5.2 硬件资料
6. SDK 工程目录介绍
7. SDK交叉编译工具链介绍
  - 7.1 U-Boot 及Kernel编译工具链
  - 7.2 Buildroot工具链
    - 7.2.1 配置编译环境
    - 7.2.2 打包工具链
  - 7.3 Debian工具链
  - 7.4 Yocto工具链
8. SDK编译说明
  - 8.1 SDK编译命令查看
  - 8.2 SDK板级配置
  - 8.3 SDK定制化配置
  - 8.4 全自动编译
  - 8.5 各模块编译
    - 8.5.1 U-Boot编译
    - 8.5.2 Kernel编译
    - 8.5.3 Recovery编译
    - 8.5.4 Buildroot编译

- 8.5.4.1 Buildroot模块编译
  - 8.5.5 Debian编译
  - 8.5.6 Yocto 编译
  - 8.6 固件的打包
- 9. 刷机说明
  - 9.1 Windows 刷机说明
  - 9.2 Linux 刷机说明
  - 9.3 系统分区说明
- 10. 常见问题解答 (FAQ)
  - 10.1 如何开启 DDR 变频
  - 10.2 RV1126BP 无法读写的问题
  - 10.3 如何读取CPU序列号

# 1. SDK预编译镜像

使用RV1126B Linux SDK预编译镜像，开发人员可以省去从源代码编译整个操作系统的过程，直接将预编译的镜像刷入RV1126B开发板，从而快速启动开发和进行相关评估、对比，可减少因编译问题导致的开发时间和成本浪费。

可从公开地址下载 [SDK固件下载点击这里](#)

固件路径：**通用 Linux SDK 固件-> Linux6.1 -> RV1126B**

如果需要修改SDK代码或快速入门，请参考下面章节。

## 2. 开发环境搭建

### 2.1 准备开发环境

我们推荐使用 Ubuntu 22.04 或更高版本的系统进行编译。其他的 Linux 版本可能需要对软件包做相应调整。除了系统要求外，还有其他软硬件方面的要求。

硬件要求：64 位系统，硬盘空间大于 40G。如果您进行多个构建，将需要更大的硬盘空间。

软件要求：Ubuntu 22.04 或更高版本系统。

### 2.2 安装库和工具集

使用命令行进行设备开发时，可以通过以下步骤安装编译SDK需要的库和工具。

使用如下apt-get命令安装后续操作所需的库和工具：

```
sudo apt-get update && sudo apt-get install git ssh make gcc libssl-dev \
liblz4-tool expect expect-dev g++ patchelf chrpath gawk texinfo chrpath \
diffstat binfmt-support qemu-user-static live-build bison flex fakeroot \
cmake gcc-multilib g++-multilib unzip device-tree-compiler ncurses-dev \
libgucharmap-2-90-dev bzip2 expat gpgv2 cpp-aarch64-linux-gnu libgmp-dev \
libmpc-dev bc python-is-python3 python2 gettext libc6-dev libncurses-dev rsync
```

#### 说明：

安装命令适用于Ubuntu22.04，其他版本请根据安装包名称采用对应的安装命令，若编译遇到报错，可以视报错信息，安装对应的软件包。其中：

- python要求安装python 3.6及以上版本，此处以python 3.6为例。
- make要求安装 make 4.0及以上版本，此处以 make 4.2为例。
- lz4要求安装 lz4 1.7.3及以上版本。
- 编译Yocto需要VPN网络。

## 2.2.1 检查和升级主机的python 版本

检查和升级主机的python 版本方法如下：

- 检查主机python 版本

```
$ python3 --version
Python 3.10.6
```

如果不满足python>=3.6版本的要求， 可通过如下方式升级：

- 升级python 3.6.15 新版本

```
PYTHON3_VER=3.6.15
echo "wget https://www.python.org/ftp/python/${PYTHON3_VER}/Python-${PYTHON3_VER}.tgz"
echo "tar xf Python-${PYTHON3_VER}.tgz"
echo "cd Python-${PYTHON3_VER}"
echo "sudo apt-get install libsqlite3-dev"
echo "./configure --enable-optimizations"
echo "sudo make install -j8"
```

## 2.2.2 检查和升级主机的make 版本

检查和升级主机的make 版本方法如下：

- 检查主机make 版本

```
$ make -v
GNU Make 4.2
Built for x86_64-pc-linux-gnu
```

- 升级make 4.2 新版本

```
$ sudo apt update && sudo apt install -y autoconf autopoint

git clone https://gitee.com/mirrors/make.git
cd make
git checkout 4.2
git am $BUILDR00T_DIR/package/make/*.patch
autoreconf -f -i
./configure
make make -j8
sudo install -m 0755 make /usr/bin/make
```

### 2.2.3 检查和升级主机的 lz4 版本

检查和升级主机的 lz4 版本方法如下：

- 检查主机 lz4 版本

```
$ lz4 -v
*** LZ4 command line interface 64-bits v1.9.3, by Yann Collet ***
```

- 升级 lz4 新版本

```
git clone https://gitee.com/mirrors/LZ4_old1.git
cd LZ4_old1

make
sudo make install
sudo install -m 0755 lz4 /usr/bin/lz4
```

## 3. Docker环境搭建

为帮助开发者快速完成上面复杂的开发环境准备工作，我们也提供了交叉编译器Docker镜像，以便客户可以快速验证，从而缩短编译环境的构建时间。

使用Docker环境前，可参考如下文档进行操作

<SDK>/docs/cn/Linux/Docker/Rockchip\_Developer\_Guide\_Linux\_Docker\_Deploy\_CN.pdf。

已验证的系统如下：

| 发行版本         | Docker 版本 | 镜像加载 | 固件编译         |
|--------------|-----------|------|--------------|
| ubuntu 22.04 | 24.0.5    | pass | pass         |
| ubuntu 21.10 | 20.10.12  | pass | pass         |
| ubuntu 21.04 | 20.10.7   | pass | pass         |
| ubuntu 18.04 | 20.10.7   | pass | pass         |
| fedora35     | 20.10.12  | pass | NR (not run) |

Docker镜像可从网址 [Docker镜像](#) 获取。

## 4. 软件开发指南

---

### 4.1 开发向导

为帮助开发工程师快速熟悉SDK开发调试流程，随SDK同步发布

《Rockchip\_Developer\_Guide\_Linux\_Software\_CN.pdf》开发向导，该向导涵盖了芯片支持情况、Roadmap、下载与软件包获取、开发文档/工具/框架/环境搭建/版本管理/编译/分区配置/固件升级/模块开发/调试工具说明、软件及许可/开源信息、常见问题及权限申请等全流程内容。

可在 `docs/cn/RV1126B` 目录下获取，并会不断完善更新。

### 4.2 Buildroot开发指南

为帮助开发工程师更快上手熟悉Buildroot系统的开发调试，随 SDK 发布

《Rockchip\_Developer\_Guide\_Buildroot\_CN.pdf》开发指南，可在 `docs/cn/Linux/System` 目录下获取，并会不断完善更新。

### 4.3 Debian开发指南

为帮助开发工程师更快上手熟悉Debian系统的开发调试，随 SDK 发布

《Rockchip\_Developer\_Guide\_Debian\_CN.pdf》开发指南，可在 `docs/cn/Linux/System` 下获取，并会不断完善更新。

### 4.4 第三方OS移植

为帮助开发工程师更快上手熟悉第三方OS的移植适配，随 SDK 发布

《Rockchip\_Developer\_Guide\_Third\_Party\_System\_Adaptation\_CN.pdf》开发向导，可在 `docs/cn/Linux/System` 目录下获取，并会不断完善更新。

### 4.5 MCU开发指南

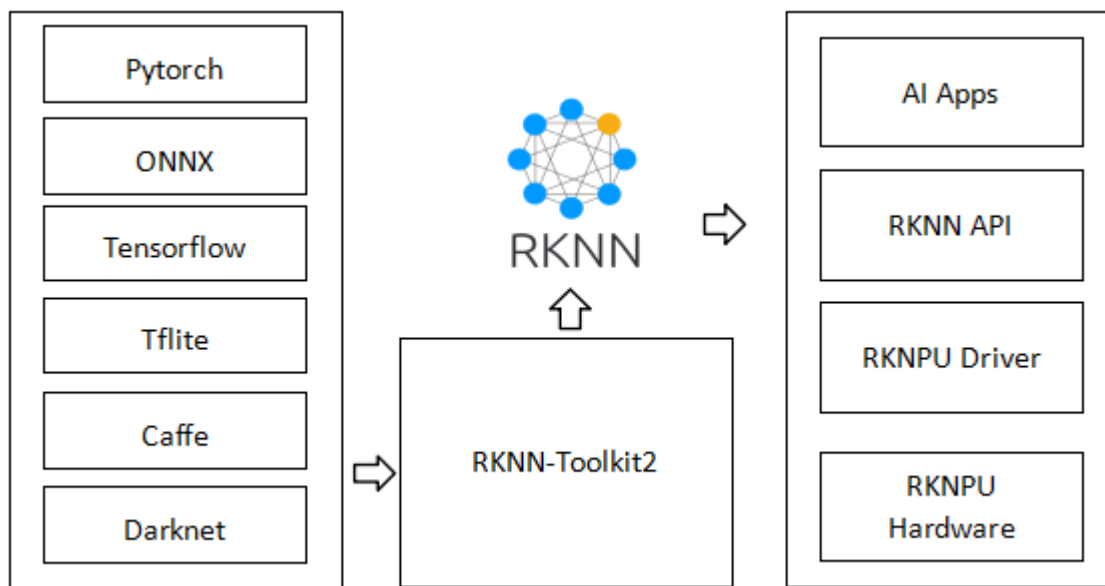
RV1126B MCU的基本使用方法，可参考如下文档：

```
<SDK>/docs/cn/RV1126B/Quick-start/Rockchip_RV1126B_Quick_Start_RT-Thread_MCU_CN.pdf
```



## 4.6 RKNN开发指南

RKNN软件栈可帮助用户快速将AI模型部署至Rockchip芯片。整体框架如下：



使用RKNPU前，用户需先在计算机运行RKNN-Toolkit2工具，将训练好的模型转换为RKNN格式模型，随后通过RKNN C API或Python API在开发板上进行推理。

### 4.6.1 RKNN-TOOLKIT2

RKNN-Toolkit2是在PC上进行RKNN模型生成及性能评估的开发套件。

RKNN-Toolkit-Lite2是Rockchip NPU平台提供Python编程接口，协助用户部署RKNN模型并加速AI应用实现。

开发套件在 `external/rknn-toolkit2` 目录下，主要用来实现模型转换、优化、量化、推理、性能评估和精度分析等一系列功能。

基本功能如下：

| 功能   | 说明                                                                                                                                          |
|------|---------------------------------------------------------------------------------------------------------------------------------------------|
| 模型转换 | 支持Pytorch / TensorFlow / TFLite / ONNX / Caffe / Darknet的浮点模型<br>支持Pytorch / TensorFlow / TFLite的量化感知模型（QAT）<br>支持动态输入模型（动态化/原生动态）<br>支持大模型 |
| 模型优化 | 常量折叠/ OP矫正/ OP Fuse&Convert / 权重稀疏化/ 模型剪枝                                                                                                   |
| 模型量化 | 支持量化类型：非对称i8/ fp16<br>支持Layer / Channel量化方式；Normal / KL/ MMSE量化算法<br>支持混合量化以平衡性能和精度                                                         |
| 模型推理 | 支持在PC上通过模拟器进行模型推理<br>支持将模型传到NPU硬件平台上完成模型推理（连板推理）<br>支持批量推理，支持多输入模型                                                                          |
| 模型评估 | 支持模型在NPU硬件平台上的性能和内存评估                                                                                                                       |
| 精度分析 | 支持量化精度分析功能（模拟器/ NPU）                                                                                                                        |
| 附加功能 | 支持版本/设备查询功能等                                                                                                                                |

### 4.6.2 RKNN Runtime

RKNN Runtime的开发说明在工程目录 `external/rknpu2` 下，用于推理RKNN-Toolkit2生成的rknn模型。为Rockchip NPU平台提供C/C++编程接口，助力用户部署RKNN模型并加速AI应用落地。

具体使用说明请参考当前 `doc/` 的目录文档：

```
├─ 01_Rockchip_RKNPU_Quick_Start_RKNN_SDK_V2.3.2_CN.pdf
├─ 02_Rockchip_RKNPU_User_Guide_RKNN_SDK_V2.3.2_CN.pdf
├─ 03_Rockchip_RKNPU_API_Reference_RKNN_Toolkit2_V2.3.2_CN.pdf
├─ 04_Rockchip_RKNPU_API_Reference_RKNNRT_V2.3.2_CN.pdf
├─ 05_RKNN_Compiler_Support_Operator_List_V2.3.2.pdf
├─ RKNNToolKit2_API_Difference_With_Toolkit1-2.3.2.md
├─ RKNNToolKit2_OP_Support-2.3.2.md
...
```

### 4.6.3 RKNN Driver

RKNN内核驱动负责与NPU硬件交互，已集成于Rockchip内核代码 `drivers/rknpu` 目录中。

## 4.6.4 RKNN LLM

RKNN LLM 是Rockchip面向大语言模型（Large Language Model, LLM）场景优化的推理框架，旨在帮助用户高效部署和运行大规模预训练语言模型（如 LLaMA、PaLM、ChatGLM 等）在Rockchip自研的 NPU硬件平台上。

具体使用说明请参考当前 `doc/` 的目录文档：

└─ Rockchip\_RKLLM\_SDK\_CN\_1.2.0.pdf

RKLLM大模型测试, 默认需要开启 `#include "ai/rkai.config"` 配置，比如下：

```
--- a/configs/rockchip_rv1126b_defconfig
+++ b/configs/rockchip_rv1126b_defconfig
@@ -13,6 +13,7 @@
 #include "multimedia/gst/rtsp.config"
 #include "multimedia/gst/video.config"
 #include "multimedia/mpp.config"
+#include "ai/rkai.config"
 #include "ai/npu2.config"
```

编译集成后，如下vlm通用模型测试：

`rkai_VLM_Demo /usr/share/app_conf/config/rkai_vlm_fastvlm_rv1126b.json`

```
buildroot# rkai_VLM_Demo /usr/share/app_conf/config/rkai_vlm_fastvlm_rv1126b.json
rkai vlm version: 0.0.1
=====Config Loaded=====
vision_model_path: /usr/share/app_conf/models/fastvlm-0.5b-fp16.rv1126b.rknn
llm_model_path: /usr/share/app_conf/models/fastvlm-0.5B-w8a8_level0.rv1126b.rkllm
  input_path: /usr/share/app_conf/inputs/image_1024x1204.npy
  max_context_len: 512
  max_new_tokens: 128
    top_k: 1
skip_special_token: 1
  base_domain_id:0
    img_start:<|im_start|>
    img_end:<|im_end|>
    img_content:<|im_content|>
=====
model input num: 1, output num: 1
===== Inputs Attribute =====
Index      : 0
Name       : images
Num Dims    : 4
Dims       : [1, 1024, 1024, 3]
Num Elems  : 3145728
Size       : 6291456 bytes
Format     : NHWC
Type       : FP16
Qnt Type   : AFFINE
Zero Point : 0
Scale     : 1.000000
```

=====

===== Outputs Attribute =====

Index : 0  
Name : projector\_features  
Num Dims : 3  
Dims : [1, 256, 896]  
Num Elems : 229376  
Size : 458752 bytes  
Format : UNDEFINED  
Type : FP16  
Qnt Type : AFFINE  
Zero Point : 0  
Scale : 1.000000

=====

model input height=1024, width=1024, channel=3,out\_elems=229376  
main: Model Vision loaded in 2329.10 ms  
I rkllm: rkllm-runtime version: 1.2.2, rknpu driver version: 0.9.8, platform: RV1126B  
I rkllm: loading rkllm model from /usr/share/app\_conf/models/fastvlm-0.5B-w8a8\_level0.rv1126b.rkllm  
I rkllm: rkllm-toolkit version: 1.2.1, max\_context\_limit: 4096, npu\_core\_num: 1, target\_platform: RV1126B, model\_dtype: W8A8  
I rkllm: Enabled cpus: [0, 1, 2, 3]  
I rkllm: Enabled cpus num: 4  
rkllm init success  
main: Model LLM loaded in 4722.97 ms  
Loading /usr/share/app\_conf/inputs/image\_1024x1204.npy  
npy data type:uint8  
rkai\_ctx->img\_elems: 229376  
main: Model run in 2744.84 ms

\*\*\*\*\*可输入以下问题对应序号获取回答/或自定义输入\*\*\*\*\*

[0] <image>What is in the image?  
[1] <image>这张图片中有什么元素?

\*\*\*\*\*

user: 0  
<image>What is in the image?  
I rkllm: reset chat template:  
I rkllm: system\_prompt: <|im\_start|>system\nYou are a helpful assistant.<|im\_end|>\n  
I rkllm: prompt\_prefix: <|im\_start|>user\n  
I rkllm: prompt\_postfix: <|im\_end|>\n<|im\_start|>assistant\n  
W rkllm: Calling rkllm\_set\_chat\_template will disable the internal automatic chat template parsing, including enable\_thinking. Make sure your custom prompt is complete.

user: <image>What is in the image?  
robot: In the image, there is an astronaut on the moon.

|         |      |      |
|---------|------|------|
| +-----+ |      |      |
| Stage   | Perf |      |
| +-----+ |      |      |
| Vision  | 2745 | (ms) |

```
| TTFT          | 1682.863      (ms) |
| TPS           | 13.074 (tokens/s) |
+-----+-----+
```

## 4.7 实时性Linux开发说明

随着产品对实时性能要求的提高，标准Linux的实时性已经满足不了很多产品的需求，需要对标准Linux进行一定的优化，来提高实时性能，比如：PREEMPT\_RT，Xenomai实时系统。

下面是进行是基于RV1126B Buildroot上分别基于PREEMPT\_RT，Xenomai进行压力测试的延迟情况，具体如下：

压力测试：

```
stress-ng -c 4 --io 2 --vm 2 --vm-bytes 256M --timeout 1000000s
```

### 4.7.0.1 PREEMPT\_RT Patch

```
root@rv1126b-buildroot:/# cyclicttest -c 0 -m -a -t 4 -p 99 -D 24h
```

```
Apolicy: fifo: loadavg: 10.80 11.74 12.03 11/232 3403
```

|              |      |        |            |      |   |      |   |      |   |      |    |
|--------------|------|--------|------------|------|---|------|---|------|---|------|----|
| T: 0 ( 1915) | P:99 | I:1000 | C:82647713 | Min: | 0 | Act: | 3 | Avg: | 2 | Max: | 38 |
| T: 1 ( 1916) | P:99 | I:1000 | C:82647712 | Min: | 0 | Act: | 2 | Avg: | 2 | Max: | 43 |
| T: 2 ( 1917) | P:99 | I:1000 | C:82647709 | Min: | 0 | Act: | 3 | Avg: | 2 | Max: | 47 |
| T: 3 ( 1918) | P:99 | I:1000 | C:82647698 | Min: | 0 | Act: | 3 | Avg: | 3 | Max: | 58 |

(测试24小时)

### 4.7.0.2 Xenomai Cobalt Mode

```
root@rv1126b-buildroot:/# echo 0 > /proc/xenomai/latency
```

```
root@rv1126b-buildroot:/# ./usr/demo/cyclicttest -m -n -c 0 -p99 -t -D 24h
```

```
Apolicy: fifo: loadavg: 9.06 8.88 8.84 9/166 3460
```

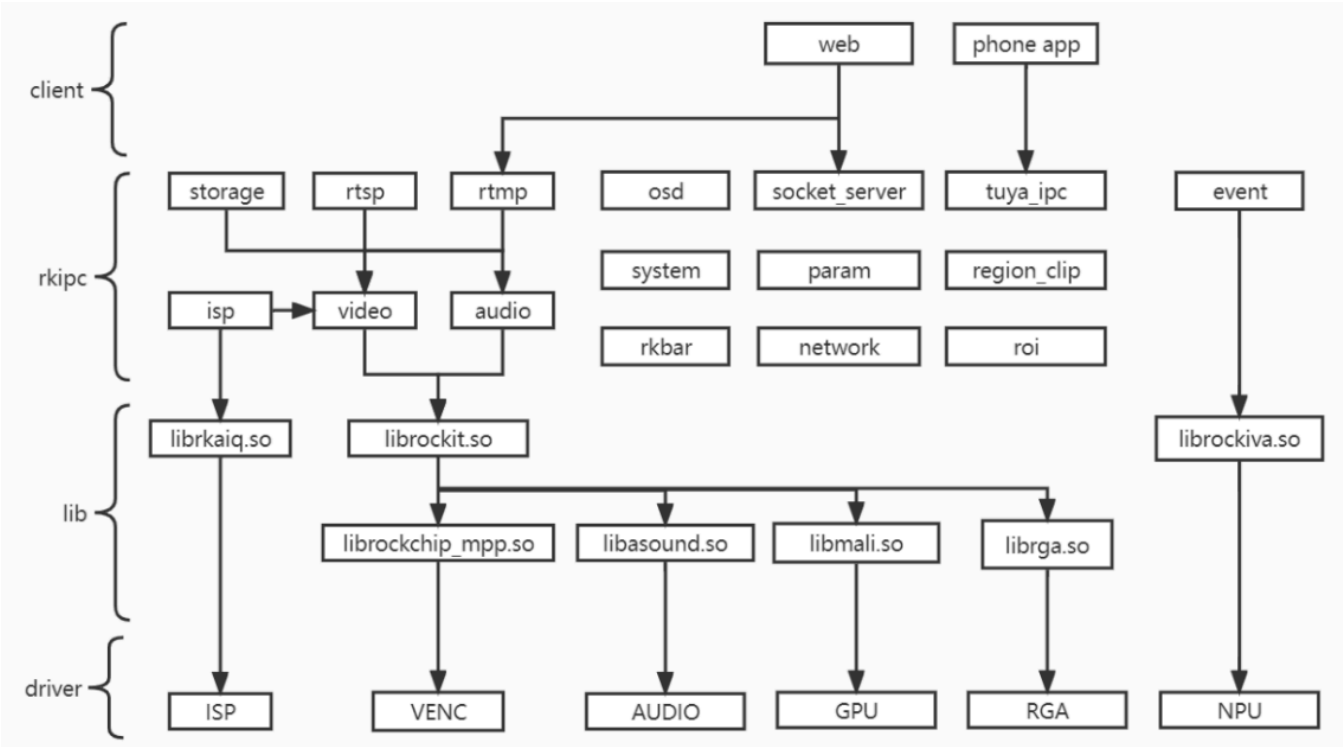
|              |      |        |            |      |   |      |   |      |   |      |    |
|--------------|------|--------|------------|------|---|------|---|------|---|------|----|
| T: 0 ( 1837) | P:99 | I:1000 | C:86399995 | Min: | 2 | Act: | 6 | Avg: | 7 | Max: | 91 |
| T: 1 ( 1838) | P:99 | I:1000 | C:86399979 | Min: | 2 | Act: | 8 | Avg: | 8 | Max: | 70 |
| T: 2 ( 1839) | P:99 | I:1000 | C:86399963 | Min: | 2 | Act: | 8 | Avg: | 6 | Max: | 41 |
| T: 3 ( 1840) | P:99 | I:1000 | C:86399939 | Min: | 2 | Act: | 8 | Avg: | 9 | Max: | 72 |

(测试24小时)

详情请参考 [docs/Patches/Real-Time-Performance/](#) 开发补丁包和说明。

## 4.8 RKIPC开发指南

针对RV1126B平台的单目IPC产品，支持网页和rtsp/rtmp预览，参数动态修改。RKIPC整体框架如下：



RKIPC应用开发指南详见如下文档：

[docs/cn/Linux/ApplicationNote/Rockchip\\_Developer\\_Guide\\_Linux\\_RKIPC\\_CN.pdf](#)。

## 4.9 AVS 多目拼接

AVS（Any View Stitching）算法支持将多路输入图像实时拼接为一幅图像，有效扩展整体视野范围。

- **API 参考：** [docs/cn/Socs/RV1126B/RKIPC/Rockchip\\_Developer\\_Guide\\_MPI\\_C\\_CN.pdf](#) 中的 AVS 章节。
- **标定工具下载：** <https://console.zbox.filez.com/l/g0a8F8>

## 4.10 AI-ISP 图像增强

AI-ISP 基于深度学习优化图像质量，涵盖噪声抑制、清晰度、对比度、亮度及色彩增强等功能。

RV1126B集成专用 AI-ISP 硬件，突破传统方案的算力瓶颈，不占用通用 NPU 资源。

- **所需文件：** `<SDK>/external/camera_engine_rkaiq/rkaiq/iqfiles/isp35/ainr` 目录下的配置及模型文件。
- **API 及调试参考：** [docs/cn/Socs/RV1126B/RKIPC/Rockchip\\_Tuning\\_Guide\\_ISP35\\_CN.pdf](#) 中的 aibnr 章节。

## 4.11 AI-Remosaic 像素重排

通过专用 AI-ISP 硬件将 Quad Bayer/4CELL 传感器数据实时转换为传统 Bayer 阵列，实现日夜双模自适应高清成像。

- **所需文件：** <SDK>/external/camera\_engine\_rkaiq/rkaiq/iqfiles/isp35/airms 中的配置及模型文件。
- **传感器支持：** 需支持 4CELL 格式输出，可参考 <SDK>/kernel/drivers/media/i2c/os12d40.c。

## 4.12 FEC 鱼眼畸变校正

用于校正鱼镜头产生的图像畸变，还原更接近人眼视角的图像。

- **开发参考：**
  - docs/cn/Socs/RV1126B/RKIPC/Rockchip\_Developer\_Guide\_MPI\_C\_CN.pdf 中的 GDC 章节
  - docs/cn/Socs/RV1126B/RKIPC/Rockchip\_Development\_Guide\_FEC\_CN.pdf

## 4.13 软件更新记录

- 软件发布版本升级通过工程 xml 进行查看，具体方法如下：

```
.repo/manifests$ realpath rv1126b_linux6.1_release.xml
# 例如:打印的版本号为v1.0.0，更新时间为20250620
# <SDK>/ .repo/manifests/release/rv1126b_linux6.1_release_v1.0.0_20250620.xml
```

- 软件发布版本升级更新内容通过工程文本可以查看，参考工程目录：

```
<SDK>/ .repo/manifests/RV1126B_Linux6.1_SDK_Note.md
或者
<SDK>/docs/cn/RV1126B/RV1126B_Linux6.1_SDK_Note.md
```

- 板端可通过如下方式获取版本信息

```
buildroot:/# cat /etc/os-release
NAME=Buildroot
VERSION=linux-6.1-stan-rkr6
ID=buildroot
VERSION_ID=2024.02
PRETTY_NAME="Buildroot 2024.02"
OS="buildroot"
RK_BUILD_INFO="XXX Mon Jun 16 17:11:18 CST 2025 - rockchip_rv1126b_ipc"
```

## 5. 硬件开发指南

### 5.1 芯片资料

为帮助开发工程师更快上手熟悉 RV1126B 的开发调试工作，随 SDK 发布《RV1126B\_brief\_datasheet\_V1.0\_20241219.pdf》和《Rockchip\_RV1126B\_Datasheet\_\*.pdf》芯片手册。可在 docs/cn/RV1126B/Datasheet 目录下获取，并会不断完善更新。

### 5.2 硬件资料

硬件相关开发可以参考用户使用指南，在工程目录 <SDK>/docs/cn/RV1126B/Hardware/：

```
<SDK>/docs/cn/RV1126B/Hardware/
├── RV1126&RV1126B-
│   └── P_Differences_Description_and_Compatibility_Design_Notes_V1.26_20250604_CN&EN.pdf
├── RV1126&RV1126B_Function_Mapping_Table_V1.0_20250527.xlsx
├── RV1126B-P_Package_PinLength_V1.0_20250318_EN.xlsx
├── RV1126B-P_PinOut_V1.0_20250519_EN.xlsx
├── RV1126B_EVB1_User_Guide_V1.0_20250613_CN.pdf
├── RV1126B_Hardware_Design_Guide_V1.0_20250828_CN.pdf
├── RV1126B_Methods_for_Processing_Unused_Pins_V1.0_20250606_EN.xlsx
├── RV1126B_Package_PinLength_V1.0_20250318_EN.xlsx
├── RV1126B_PinOut_V1.1_20250519_EN.xlsx
└── RV1126B_Power_Consumption_Test_Report_V1.0_20250527_CN.xlsx
```

## 6. SDK 工程目录介绍

SDK 工程目录包含有 buildroot、debian、yocto、app、kernel、u-boot、device、docs、external、prebuilts等目录。采用manifest来管理仓库，用repo工具来管理每个目录或其子目录会对应的 git 工程。

- app：存放上层应用 APP，主要是一些应用Demo。
- buildroot：基于 Buildroot（2024）开发的根文件系统。
- debian：基于 Debian bookworm（12）开发的根文件系统。
- device/rockchip：存放芯片板级配置以及SDK编译和打包固件的脚本和文件等。
- docs：存放通用开发指导文档、芯片平台相关文档、Linux 系统开发相关文档、其他参考文档等。
- external：存放第三方相关仓库，包括NPU、算法、音视频、摄像头、网络、安全等。
- kernel：存放 Kernel 开发的代码。
- output：存放每次生成的固件信息、编译信息、XML、主机环境等。
- prebuilts：存放交叉编译工具链。
- rkbin：存放 Rockchip 相关二进制和工具。



- rockdev：存放编译输出固件，实际软链接到 `output/firmware`。
- tools：存放 Linux 和 Window 操作系统下常用工具。
- u-boot：存放基于 v2017.09 版本进行开发的 U-Boot 代码。
- yocto：存放基于 Yocto scarthgap (5.0) 开发的根文件系统。

## 7. SDK交叉编译工具链介绍

鉴于Rockchip Linux SDK目前只在Linux PC环境下编译，我们也仅提供了Linux下的交叉编译工具链。prebuilt目录下的预置的工具链是给U-Boot和Kernel使用。具体Rootfs需要用各自对应的工具链，或者使用第三方工具链进行编译。

### 7.1 U-Boot 及Kernel编译工具链

SDK prebuilts目录预置交叉编译目前用于U-Boot 及Kernel编译，如下：

| 目录                                                                                 | 说明                    |
|------------------------------------------------------------------------------------|-----------------------|
| prebuilts/gcc/linux-x86/aarch64/gcc-arm-10.3-2021.07-x86_64-aarch64-none-linux-gnu | gcc arm 10.3.1 64位工具链 |
| prebuilts/gcc/linux-x86/arm/gcc-arm-10.3-2021.07-x86_64-arm-none-linux-gnueabi     | gcc arm 10.3.1 32位工具链 |

可从以下地址下载工具链：

[点击这里](#)

### 7.2 Buildroot工具链

#### 7.2.1 配置编译环境

若需要编译单个模块或者第三方应用，需交叉编译环境进行配置。比如RV1126B其交叉编译工具位于 `buildroot/output/rockchip_rv1126b/host/usr` 目录下，需要将工具的bin/目录和 `aarch64-buildroot-linux-gnu/bin/` 目录设为环境变量，在顶层目录执行自动配置环境变量的脚本：

```
source buildroot/envsetup.sh rockchip_rv1126b
```

输入命令查看：

```
cd buildroot/output/rockchip_rv1126b/host/usr/bin
./aarch64-linux-gcc --version
```

此时会打印如下信息：

```
aarch64-linux-gcc.br_real (Buildroot -gc307c95550) 13.3.0
```

## 7.2.2 打包工具链

Buildroot 支持将内置工具链打包为压缩包，以供第三方应用单独编译使用。有关如何打包工具链的详细信息，请参阅 Buildroot 官方文档：

```
buildroot/docs/manual/using-buildroot-toolchain.txt
```

在 SDK 中，可以直接运行以下命令来生成工具链包：

```
./build.sh bmake: sdk
```

生成的工具链包位于 `buildroot/output/*/images/` 目录，名为 `aarch64-buildroot-linux-gnu_sdk-buildroot.tar.gz`，供有需求的用户使用。解压后，`gcc` 的路径将是：

```
./aarch64-buildroot-linux-gnu_sdk-buildroot/bin/aarch64-buildroot-linux-gnu-gcc
```

可从以下地址下载工具链：

[点击这里](#)

## 7.3 Debian工具链

使用docker机器端，`gcc`或者 `dpkg-buildpackage` 进行相关编译。

## 7.4 Yocto工具链

参考如下：

```
https://wiki.yoctoproject.org/wiki/Building_your_own_recipes_from_first_principles#Adding_new_recipes_to_the_build_system
https://docs.yoctoproject.org/dev/dev-manual/new-recipe.html
```

# 8. SDK编译说明

SDK可通过 `make` 或 `./build.sh` 加目标参数进行相关功能的配置和编译。

具体参考 `device/rockchip/common/README.md` 编译说明。

为了确保SDK每次更新都能顺利进行，建议在更新前清理之前的编译产物。这样做可以避免潜在的兼容性问题或编译错误，因为旧的编译产物可能不适用于新版本的SDK。要清理这些编译产物，可以直接运行命令 `./build.sh cleanall`。

一键编译

| 芯片        | 类型  | ARM<br>架构版本 | 权限<br>类型 | 参考机型          | 一键编译                                                                             | rootfs 编译            | kernel 编译                                                                                                                                                               | uboot 编译                                                                                        |
|-----------|-----|-------------|----------|---------------|----------------------------------------------------------------------------------|----------------------|-------------------------------------------------------------------------------------------------------------------------------------------------------------------------|-------------------------------------------------------------------------------------------------|
| RV1126B   | 行业  | 64 位        | 读写       | RV1126B EVB1  | ./build.sh lunch:rockchip_rv1126b-evb1-v10_defconfig &&<br>./build.sh            | ./build.sh<br>rootfs | make ARCH=arm64<br>rockchip_linux_defconfig rv1126b.config;<br>make ARCH=arm64 rv1126b-evb1-v10.img<br>BOOT_ITS=boot.its -j24                                           | ./make.sh rv1126b --spl-new                                                                     |
| RV1126B   | 工业  | 64 位        | 读写       | RV1126B EVB4  | ./build.sh lunch:rockchip_rv1126b-evb4-v10_defconfig &&<br>./build.sh            | ./build.sh<br>rootfs | make ARCH=arm64<br>rockchip_linux_defconfig rv1126b.config;<br>make ARCH=arm64 rv1126b-evb4-v10.img<br>BOOT_ITS=boot.its -j24                                           | ./make.sh rv1126b --spl-new                                                                     |
| RV1126B   | 快启  | 64 位        | 只读       | RV1126B EVB1  | ./build.sh lunch:rockchip_rv1126b-fastboot_defconfig &&<br>./build.sh            | ./build.sh<br>rootfs | make ARCH=arm64 rv1126b_defconfig<br>rv1126b.config rv1126b-fastboot.config;<br>make ARCH=arm64 rv1126b-evb1-v10-<br>fastboot-emmc.img BOOT_ITS=boot.its -<br>j24       | ./make.sh rv1126b-emmc-fastboot rk-gpt --spl-new                                                |
| RV1126B   | IPC | 32 位        | 读写       | RV1126B EVB1  | ./build.sh<br>lunch:rockchip_rv1126b_ipc_32-evb1-v10_defconfig &&<br>./build.sh  | ./build.sh<br>rootfs | make ARCH=arm rv1126b_linux_defconfig;<br>make ARCH=arm rv1126b-evb1-v10.img<br>BOOT_ITS=boot.its -j24                                                                  | ./make.sh rv1126b --spl-new                                                                     |
| RV1126B   | IPC | 64 位        | 读写       | RV1126B EVB1  | ./build.sh<br>lunch:rockchip_rv1126b_ipc_64-evb1-v10_defconfig &&<br>./build.sh  | ./build.sh<br>rootfs | make ARCH=arm64 rv1126b_defconfig<br>rv1126b.config;<br>make ARCH=arm64 rv1126b-evb1-v10.img<br>BOOT_ITS=boot.its -j24                                                  | ./make.sh rv1126b --spl-new                                                                     |
| RV1126B   | 扫地机 | 64 位        | 读写       | RV1126B EVB1  | ./build.sh lunch:rockchip_rv1126b-robot_defconfig &&<br>./build.sh               | ./build.sh<br>rootfs | make ARCH=arm64 rv1126b_defconfig<br>rv1126b.config rv1126b-robot.config<br>rockchip_amp.config;<br>make ARCH=arm64 rv1126b-evb1-v10-<br>amp.img BOOT_ITS=boot.its -j24 | ./make.sh rv1126b rk-amp --spl-new                                                              |
| RV1126B-P | 行业  | 64 位        | 读写       | RV1126BP EVB1 | ./build.sh lunch:rockchip_rv1126bp-evb1-v10_defconfig<br>&& ./build.sh           | ./build.sh<br>rootfs | make ARCH=arm64<br>rockchip_linux_defconfig rv1126b.config;<br>make ARCH=arm64 rv1126bp-evb1-<br>v12.img BOOT_ITS=boot.its -j24                                         | ./make.sh rv1126b ./rkbin/RKBOOT/RV1126BPMINIALL.ini --<br>spl-new                              |
| RV1126B-P | 快启  | 64 位        | 只读       | RV1126BP EVB1 | ./build.sh lunch:rockchip_rv1126bp-fastboot_defconfig &&<br>./build.sh           | ./build.sh<br>rootfs | make ARCH=arm64 rv1126b_defconfig<br>rv1126b.config rv1126b-fastboot.config;<br>make ARCH=arm64 rv1126bp-evb1-v12-<br>fastboot-emmc.img BOOT_ITS=boot.its -<br>j24      | ./make.sh rv1126b-emmc-fastboot rk-gpt<br>./rkbin/RKBOOT/RV1126BPMINIALL_FASTBOOT.ini --spl-new |
| RV1126B-P | IPC | 32 位        | 读写       | RV1126BP EVB1 | ./build.sh<br>lunch:rockchip_rv1126bp_ipc_32-evb1-v10_defconfig &&<br>./build.sh | ./build.sh<br>rootfs | ARCH=arm rv1126b_linux_defconfig;<br>make ARCH=arm rv1126bp-evb1-v12.img<br>BOOT_ITS=boot.its -j24                                                                      | ./make.sh rv1126b ./rkbin/RKBOOT/RV1126BPMINIALL.ini --<br>spl-new                              |
| RV1126B-P | IPC | 64 位        | 读写       | RV1126BP EVB1 | ./build.sh<br>lunch:rockchip_rv1126bp_ipc_64-evb1-v10_defconfig &&<br>./build.sh | ./build.sh<br>rootfs | make ARCH=arm64 rv1126b_defconfig<br>rv1126b.config;<br>make ARCH=arm64 rv1126bp-evb1-<br>v12.img BOOT_ITS=boot.its -j24                                                | ./make.sh rv1126b ./rkbin/RKBOOT/RV1126BPMINIALL.ini --<br>spl-new                              |
| RV1126B-P | 扫地机 | 64 位        | 读写       | RV1126BP EVB1 | ./build.sh lunch:rockchip_rv1126bp-robot_defconfig &&<br>./build.sh              | ./build.sh<br>rootfs | make ARCH=arm64 rv1126b_defconfig<br>rv1126b.config rv1126b-robot.config;<br>make ARCH=arm64 rv1126bp-evb1-<br>v12.img BOOT_ITS=boot.its -j24                           | ./make.sh rv1126b ./rkbin/RKBOOT/RV1126BPMINIALL.ini --<br>spl-new                              |

注意：

• **Rootfs编译：**

默认是编译Buildroot系统，如果需要其他系统，设置相应环境变量即可。比如编译buildroot系统: RK\_ROOTFS\_SYSTEM=buildroot ./build.sh  
编译debian系统: RK\_ROOTFS\_SYSTEM=debian ./build.sh  
编译yocto系统: RK\_ROOTFS\_SYSTEM=yocto ./build.sh

• **Kernel、U-boot编译：**其中需要制指定工具链。

• 比如SDK内置32位工具链：  
export CROSS\_COMPILE=../prebuilts/gcc/linux-x86/arm/gcc-arm-10.3-2021.07-x86\_64-arm-none-linux-gnueabi/bin/arm-none-linux-gnueabi-

• 比如SDK内置64位工具链：  
export CROSS\_COMPILE=../prebuilts/gcc/linux-x86/aarch64/gcc-arm-10.3-2021.07-x86\_64-aarch64-none-linux-gnu/bin/aarch64-none-linux-gnu-

## 8.1 SDK编译命令查看

`make help` , 例如:

```
$ make help
menuconfig          - interactive curses-based configurator
oldconfig           - resolve any unresolved symbols in .config
synconfig           - Same as oldconfig, but quietly, additionally update deps
olddefconfig        - Same as synconfig but sets new symbols to their default value
savedefconfig       - Save current config to RK_DEFCONFIG (minimal config)
...
```

make实际运行是 `./build.sh`

即也可运行 `./build.sh <target>` 来编译相关功能, 具体可通过 `./build.sh help` 查看具体编译命令。

```
./build.sh -h
```

```
##### Rockchip Linux SDK #####
```

```
Manifest: rv1126b_linux6.1_release_v1.0.0_20250620.xml
```

```
Log colors: message notice warning error fatal
```

```
sage: build.sh [OPTIONS]
```

```
Available options:
```

```
chip[:<chip>[:<config>]]          choose chip
```

```
defconfig[:<config>]              choose defconfig
```

```
*_defconfig                        switch to specified defconfig
```

```
    available defconfigs:
```

```
    rockchip_defconfig
```

```
    rockchip_rv1126b_evb1_v10_defconfig
```

```
    rockchip_rv1126b_evb4_v10_defconfig
```

```
    rockchip_rv1126b_ipc_32_evb1_v10_defconfig
```

```
    rockchip_rv1126b_ipc_64_evb1_v10_defconfig
```

```
    rockchip_rv1126b_robot_defconfig
```

```
    rockchip_rv1126bp_evb1_v10_defconfig
```

```
    rockchip_rv1126bp_ipc_32_evb1_v10_defconfig
```

```
    rockchip_rv1126bp_ipc_64_evb1_v10_defconfig
```

```
    rockchip_rv1126bp_robot_defconfig
```

```
olddefconfig                      resolve any unresolved symbols in .config
```

```
savedefconfig                     save current config to defconfig
```

```
menuconfig                        interactive curses-based configurator
```

```
config                            modify SDK defconfig
```

```
print-parts                       print partitions
```

```
list-parts                        alias of print-parts
```

```
edit-parts                        edit raw partitions
```

```
misc                              pack misc image
```

```
kernel-6.1[:dry-run]             build kernel 6.1
```

```
kernel[:dry-run]                 build kernel
```

```
recovery-kernel[:dry-run]        build kernel for recovery
```

|                                    |                                               |
|------------------------------------|-----------------------------------------------|
| kernel-modules[:<dst dir>:dry-run] | build kernel modules                          |
| modules[:<dst dir>:dry-run]        | alias of kernel-modules                       |
| linux-headers[:<arch>:dry-run]     | build linux-headers                           |
| kernel-config[:dry-run]            | modify kernel defconfig                       |
| kconfig[:dry-run]                  | alias of kernel-config                        |
| kernel-make[:<arg1>:<arg2>]        | run kernel <b>make</b>                        |
| kmake[:<arg1>:<arg2>]              | alias of kernel-make                          |
| wifibt[:<dst dir>[:<chip>]]        | build Wifi/BT                                 |
| amp                                | build and pack amp system                     |
| buildroot-config[:<config>]        | modify buildroot defconfig                    |
| bconfig[:<config>]                 | alias of buildroot-config                     |
| buildroot-make[:<arg1>:<arg2>]     | run buildroot <b>make</b>                     |
| bmake[:<arg1>:<arg2>]              | alias of buildroot-make                       |
| buildroot-sdk                      | build the buildroot SDK tarball               |
| bsdk                               | alias of buildroot-sdk                        |
| rootfs[:<rootfs type>]             | build default rootfs                          |
| buildroot                          | build buildroot rootfs                        |
| yocto                              | build yocto rootfs                            |
| debian                             | build debian rootfs                           |
| recovery                           | build recovery                                |
| security-createkeys                | create keys <b>for</b> security               |
| security-misc                      | build misc with system encryption key         |
| security-ramboot[:system_image]    | build security ramboot                        |
| security-system[:system_image]     | build security system                         |
| security-remote-sign               | build remote signed image                     |
| loader[:dry-run]                   | build loader (u-boot)                         |
| uboot[:dry-run]                    | build u-boot                                  |
| u-boot[:dry-run]                   | alias of uboot                                |
| extra-parts                        | pack extra partition images                   |
| firmware                           | pack and check firmwares                      |
| edit-package-file                  | edit package-file                             |
| edit-ota-package-file              | edit package-file <b>for</b> OTA              |
| updateimg                          | build update image                            |
| ota-updateimg                      | build update image <b>for</b> OTA             |
| all                                | build images                                  |
| release[:<subdir>[:<name>]]        | release images and build info                 |
| all-release[:<subdir>[:<name>]]    | build and release images                      |
| shell                              | setup a shell <b>for</b> developing           |
| buildroot-shell                    | setup a shell <b>for</b> buildroot developing |
| bshell                             | alias of buildroot-shell                      |
| yocto-shell                        | setup a shell <b>for</b> yocto developing     |
| yshell                             | alias of yocto-shell                          |
| cleanall                           | cleanup all                                   |
| clean-misc                         | cleanup misc                                  |
| clean-config                       | cleanup config                                |
| clean-recovery                     | cleanup recovery                              |
| clean-amp                          | cleanup amp                                   |
| clean-kernel                       | cleanup kernel                                |
| clean-rootfs                       | cleanup rootfs                                |
| clean-firmware                     | cleanup firmware                              |
| clean-security                     | cleanup security                              |
| clean-extra-parts                  | cleanup extra-parts                           |
| clean-updateimg                    | cleanup updateimg                             |

`clean-loader`  
`post-rootfs <rootfs dir>`  
`help`  
  
Default option is 'all'.

`cleanup loader`  
`trigger post-rootfs hook scripts`  
`display this information`

8.2 SDK板级配置

进入工程 <SDK>/device/rockchip/rv1126b 目录：

| 板级配置                                        | 说明                            |
|---------------------------------------------|-------------------------------|
| rockchip_defconfig                          | 默认配置，软链接到RV1126B EVB1开发板通用配置  |
| rockchip_rv1126b_evb1_v10_defconfig         | 适用于RV1126B EVB1 板子通用配置        |
| rockchip_rv1126b_evb4_v10_defconfig         | 适用于RV1126B EVB4 板子工业配置        |
| rockchip_rv1126b_fastboot_defconfig         | 适用于RV1126B EVB1 板子快启配置        |
| rockchip_rv1126b_ipc_32_evb1_v10_defconfig  | 适用于RV1126B EVB1 板子IPC 32位配置   |
| rockchip_rv1126b_ipc_64_evb1_v10_defconfig  | 适用于RV1126B EVB1 板子IPC 64位配置   |
| rockchip_rv1126b_robot_defconfig            | 适用于RV1126B EVB1 扫地机板子配置       |
| rockchip_rv1126bp_evb1_v10_defconfig        | 适用于RV1126B-P EVB1 板子通用配置      |
| rockchip_rv1126bp_fastboot_defconfig        | 适用于RV1126B-P EVB1 板子快启配置      |
| rockchip_rv1126bp_ipc_32_evb1_v10_defconfig | 适用于RV1126B-P EVB1 板子IPC 32位配置 |
| rockchip_rv1126bp_ipc_64_evb1_v10_defconfig | 适用于RV1126B-P EVB1 板子IPC 64位配置 |
| rockchip_rv1126bp_robot_defconfig           | 适用于RV1126B-P EVB1 扫地机板子配置     |

可通过 `make lunch` 或者 `./build.sh lunch` 进行配置

```
$ ./build.sh lunch

You're building on Linux
Lunch menu...pick a combo:

1. rockchip_defconfig
2. rockchip_rv1126b_evb1_v10_defconfig
3. rockchip_rv1126b_evb4_v10_defconfig
4. rockchip_rv1126b_fastboot_defconfig
5. rockchip_rv1126b_ipc_32_evb1_v10_defconfig
6. rockchip_rv1126b_ipc_64_evb1_v10_defconfig
7. rockchip_rv1126b_robot_defconfig
8. rockchip_rv1126bp_evb1_v10_defconfig
9. rockchip_rv1126bp_fastboot_defconfig
```

```
10. rockchip_rv1126bp_ipc_32_evb1_v10_defconfig
11. rockchip_rv1126bp_ipc_64_evb1_v10_defconfig
12. rockchip_rv1126bp_robot_defconfig
Which would you like? [1]:
```

大部分客户拿到都是EVB1板子，适用通用的AIOT应用场景。

其他功能的配置可通过 `make menuconfig` 来配置相关属性。

## 8.3 SDK定制化配置

SDK可通过 `make menuconfig` 进行相关配置，目前可配组件主要如下：

```
(rockchip_rv1126b_evb1_v10_defconfig) Name of defconfig to save
[*] Rootfs (Buildroot|Debian|Yocto) --->
[*] Loader (U-Boot) --->
[ ] AMP (Asymmetric Multi-Processing System)
[*] Kernel (Embedded in an Android-style boot image) --->
    Boot (Android-style boot image) --->
[*] Recovery (based on Buildroot) --->
[ ] Security feature (secureboot, encryption, verity, etc.)
    Extra partitions (oem, userdata, etc.) --->
    Firmware (partition table, misc image, etc.) --->
[*] Update (Rockchip update image) --->
[ ] Fastboot (To optimize device boot speed) --->
    Others configurations --->
```

- Rootfs: 这里的Rootfs代表“根文件系统”，在这里可选择Buildroot、Yocto、Debian等不同的根文件系统配置。
- Loader (u-boot): 这是引导加载器的配置，通常是u-boot，用于初始化硬件并加载主操作系统。
- AMP: 多核异构启动方案，适用于需要实时性能的应用场景。
- Kernel: 这里配置内核选项，定制适合自己的硬件和应用需求的Linux内核。
- Boot: 这里配置Boot分区支持格式。
- Recovery (based on buildroot): 这是基于buildroot的recovery环境的配置，用于系统恢复和升级。
- PCBA test (based on buildroot): 这是一个基于buildroot的PCBA（印刷电路板组装）测试环境的配置。
- Security: 安全功能开启，包含Secureboot开启方式、Optee存储方式、烧写Key等。
- Extra partitions: 用于配置额外的分区。
- Firmware: 在这里配置固件相关选项。
- Update (Rockchip update image): 用于配置Rockchip完整固件的选项。
- Fastboot: 快启配置，减少 SPL 和内核阶段的系统启动时间，并跳过 U-Boot 阶段。
- Others configurations: 其他额外的配置选项。

通过 `make menuconfig` 配置界面提供了一个基于文本的用户界面来选择和配置各种选项。

配置完成后，使用 `make savedefconfig` 命令保存这些配置，这样定制化编译就会根据这些设置来进行。

通过以上config，可选择不同Rootfs/Loader/Kernel等配置，进行各种定制化编译，这样就可以灵活地选择和配置系统组件以满足特定的需求。

## 8.4 全自动编译

为了确保软件开发套件（SDK）的每次更新都能顺利进行，建议在更新前清理之前的编译产物。这样做可以避免潜在的兼容性问题或编译错误，因为旧的编译产物可能不适用于新版本的SDK。要清理这些编译产物，可以直接运行命令 `./build.sh cleanall`。

进入工程根目录执行以下命令自动完成所有的编译：

```
./build.sh all # 只编译模块代码 (u-Boot, kernel, Rootfs, Recovery)
               # 需要再执行`./build.sh ./mkfirmware.sh`进行固件打包

./build.sh     # 编译模块代码 (u-Boot, kernel, Rootfs, Recovery)
               # 打包成update.img完整升级包
               # 所有编译信息复制和生成到out目录下
```

默认是 Buildroot，可以通过设置环境变量 `RK_ROOTFS_SYSTEM` 指定不同 rootfs。`RK_ROOTFS_SYSTEM` 目前可设定三种系统：buildroot、debian、yocto。

比如需要 debain 可以通过以下命令进行生成：

```
export RK_ROOTFS_SYSTEM=debian
./build.sh
或
RK_ROOTFS_SYSTEM=debian ./build.sh
```

## 8.5 各模块编译

### 8.5.1 U-Boot编译

```
./build.sh uboot
```

### 8.5.2 Kernel编译

- 方法一

```
./build.sh kernel
```

- 方法二



```
cd kernel
export CROSS_COMPILE=../prebuilts/gcc/linux-x86/aarch64/gcc-arm-10.3-2021.07-x86_64-aarch64-
none-linux-gnu/bin/aarch64-none-linux-gnu-
make ARCH=arm64 rockchip_linux_defconfig rv1126b.config
make ARCH=arm64 rv1126b-evb1-v10.img BOOT_ITS=boot.its -j4
```

注意：`rv1126b-evb1-v10`可替换具体的硬件dts

- 方法三

```
cd kernel
export CROSS_COMPILE=aarch64-linux-gnu-
make ARCH=arm64 rockchip_linux_defconfig rv1126b.config
make ARCH=arm64 rv1126b-evb1-v10.img BOOT_ITS=boot.its -j4
```

注意：`rv1126b-evb1-v10`可替换具体的硬件dts

### 8.5.3 Recovery编译

进入工程根目录执行以下命令自动完成 Recovery 的编译及打包。

```
<SDK># ./build.sh recovery
```

编译后在 Buildroot 目录 `output/rockchip_rv1126b_recovery/images` 生成 `recovery.img`。

注：recovery.img 是包含内核，所以每次 Kernel 更改，Recovery 是需要重新打包生成。Recovery重新打包的方法如下：

```
<SDK># source buildroot/envsetup.sh
<SDK># cd buildroot
<SDK># make recovery-reconfigure
<SDK># cd -
<SDK># ./build.sh recovery
```

注：Recovery是非必需的功能，有些板级配置不会设置

### 8.5.4 Buildroot编译

进入工程目录根目录执行以下命令自动完成 Rootfs 的编译及打包：

```
./build.sh rootfs
```

编译后在Buildroot目录 `output/rockchip_rv1126b/images` 下生成不同格式的镜像, 默认使用rootfs.ext4格式。

具体可参考Buildroot开发文档参考：

`<SDK>/docs/cn/Linux/System/Rockchip_Developer_Guide_Buildroot_CN.pdf`

#### 8.5.4.1 Buildroot模块编译

可通过 `source buildroot/envsetup.sh` 来设置不同芯片和目标功能的配置

```
$ source buildroot/envsetup.sh
Top of tree: rv1126b

You're building on Linux
Lunch menu...pick a combo:

65. rockchip_rv1126b
66. rockchip_rv1126b_recovery
...

Which would you like? [1]:
```

默认选择66, `rockchip_rv1126b`。然后进入rv1126b的Buildroot目录, 开始相关模块的编译。

其中 `rockchip_rv1126b_recovery` 是用来编译Recovery模块。

比如编译 `rockchip-test` 模块, 常用相关编译命令如下:

进入 buildroot目录

```
<SDK># cd buildroot
```

- 删除并重新编译 `rockchip-test`

```
buildroot# make rockchip-test-recreate
```

- 重编 `rockchip-test`

```
buildroot# make rockchip-test-rebuild
```

- 删除 `rockchip-test`

```
buildroot# make rockchip-test-dirclean
```

或者

```
buildroot# rm -rf output/rockchip_rv1126b/build/rockchip-test-master/
```

### 8.5.5 Debian编译

```
./build.sh debian
```

编译后在 `debian` 目录下生成 `linaro-rootfs.img`。

说明：需要预先安装相关依赖包

```
sudo apt-get install binfmt-support qemu-user-static live-build
sudo dpkg -i ubuntu-build-service/packages/*
sudo apt-get install -f
```

具体可参考Debian开发文档参考：

<SDK>/docs/cn/Linux/System/Rockchip\_Developer\_Guide\_Debian\_CN.pdf

## 8.5.6 Yocto 编译

进入工程目录根目录执行以下命令自动完成 Rootfs 的编译及打包：

```
./build.sh yocto
```

编译后在 yocto 目录 build/lastest 下生成 rootfs.img。

默认用户名登录是 root。Yocto 更多信息请参考 [Rockchip Wiki](#)。

FAQ：

- 上面编译如果遇到如下问题情况：

```
Please use a locale setting which supports UTF-8 (such as LANG=en_US.UTF-8).
Python can't change the filesystem locale after loading so we need a UTF-8
when Python starts or things won't work.
```

解决方法：

```
locale-gen en_US.UTF-8
export LANG=en_US.UTF-8 LANGUAGE=en_US.en LC_ALL=en_US.UTF-8
```

或者参考 [setup-locale-python3](#)

- 如果遇到git权限问题，导致编译出错。

由于git新版本增加了CVE-2022-39253安全检测补丁，而如果旧版本的Yocto就需要poky包含如下才可修复：

```
commit ac3eb2418aa91e85c39560913c1ddfd2134555ba
Author: Robert Yang <liezhi.yang@windriver.com>
Date:   Fri Mar 24 00:09:02 2023 -0700

    bitbake: fetch/git: Fix local clone url to make it work with repo

    The "git clone /path/to/git/objects_symlink" couldn't work after the following
    change:

    https://github.com/git/git/commit/6f054f9fb3a501c35b55c65e547a244f14c38d56
```

或者通过回退PC的git版本到V2.38或早期版本也行， 比如下：

```
$ sudo apt update && sudo apt install -y libcurl4-gnutls-dev

git clone https://gitee.com/mirrors/git.git --depth 1 -b v2.38.0
cd git
make git -j8
make install
sudo install -m 0755 git /usr/bin/git
```

## 8.6 固件的打包

上面 Kernel/U-Boot/Recovery/Rootfs 各个部分的编译后，进入工程目录根目录执行以下命令自动完成所有固件打包到 `output/firmware` 目录下：

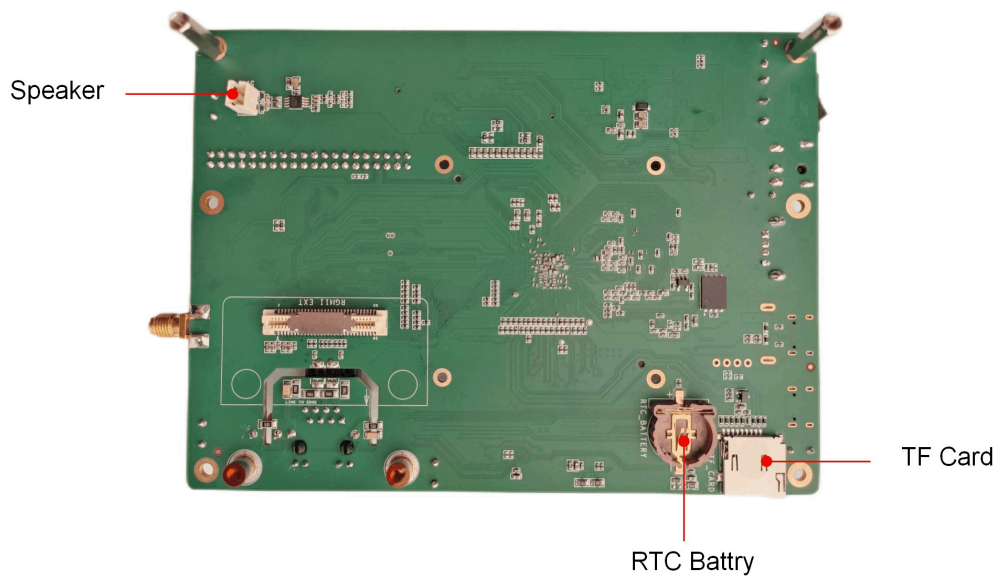
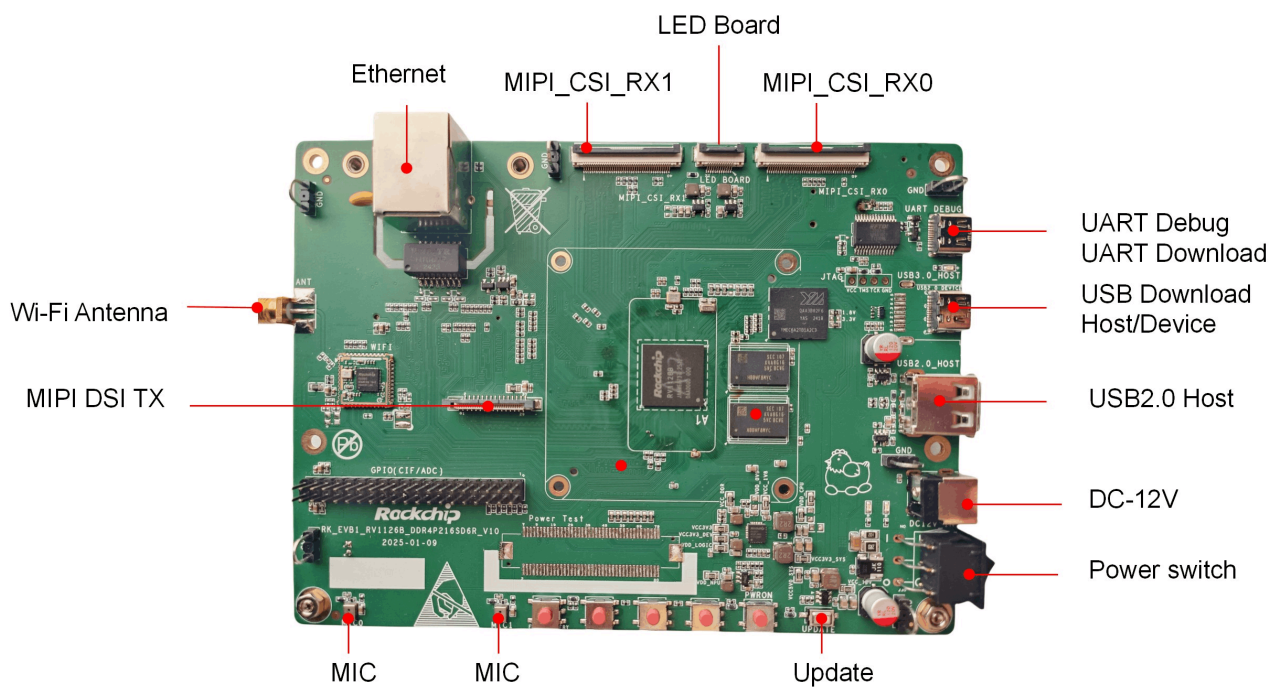
固件生成：

```
./build.sh firmware
```

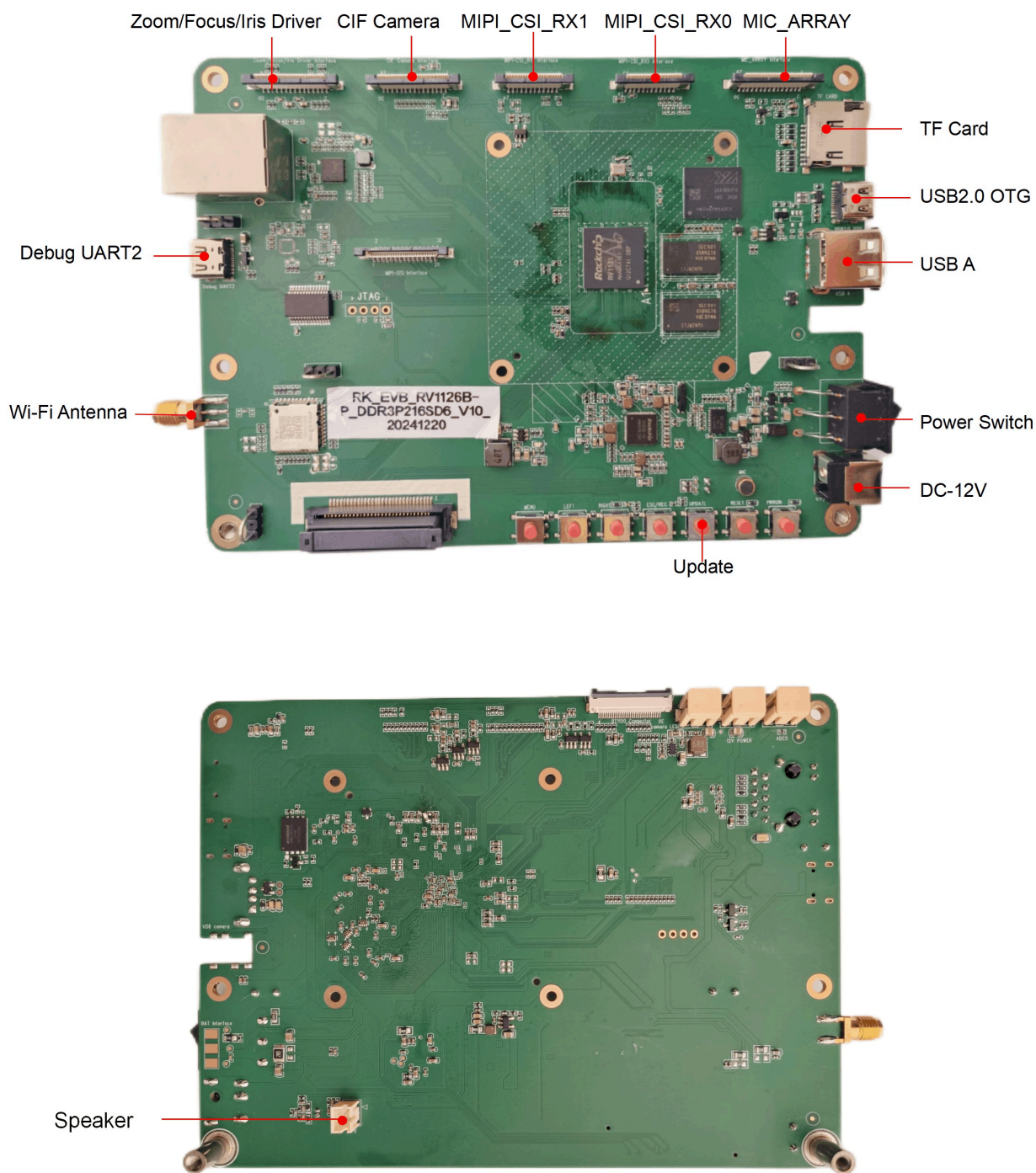
## 9. 刷机说明

---

RV1126B EVB1 V10 开发板正面接口分布图如下：



RV1126B-P EVB1 V10 开发板背面接口分布图如下：



## 9.1 Windows 刷机说明

SDK 提供 Windows 烧写工具(工具版本需要 V3.37 或以上), 工具位于工程根目录:

```
tools/  
└─ windows/RKDevTool
```



如下图，编译生成相应的固件后，设备烧写需要进入 MASKROM 或 BootROM 烧写模式，连接好 USB 下载线后，按住按键“MASKROM”不放并按下复位键“RST”后松手，就能进入MASKROM 模式，加载编译生成固件的相应路径后，点击“执行”进行烧写，也可以按 “recovery”按键不放并按下复位键 “RST” 后松手进入 loader 模式进行烧写，下面是 MASKROM 模式的分区偏移及烧写文件。

(注意： Windows PC 需要在管理员权限运行工具才可执行)

Download Image

Upgrade Firmware

Advanced Function

| #  | <input type="checkbox"/>            | Storage | Address    | Name      | Path                         | ... |
|----|-------------------------------------|---------|------------|-----------|------------------------------|-----|
| 1  | <input checked="" type="checkbox"/> |         | 0x00000000 | loader    | ..\rockdev\MiniLoaderAll.bin |     |
| 2  | <input checked="" type="checkbox"/> |         | 0x00000000 | parameter | ..\rockdev\parameter.txt     |     |
| 3  | <input checked="" type="checkbox"/> |         | 0x00004000 | uboot     | ..\rockdev\uboot.img         |     |
| 4  | <input checked="" type="checkbox"/> |         | 0x00006000 | misc      | ..\rockdev\misc.img          |     |
| 5  | <input checked="" type="checkbox"/> |         | 0x00008000 | boot      | ..\rockdev\boot.img          |     |
| 6  | <input checked="" type="checkbox"/> |         | 0x00028000 | recovery  | ..\rockdev\recovery.img      |     |
| 7  | <input type="checkbox"/>            |         | 0x00068000 | backup    |                              |     |
| 8  | <input checked="" type="checkbox"/> |         | 0x01C78000 | oem       | ..\rockdev\oem.img           |     |
| 9  | <input checked="" type="checkbox"/> |         | 0x00078000 | rootfs    | ..\rockdev\rootfs.img        |     |
| 10 | <input checked="" type="checkbox"/> |         | 0x01D18000 | userdata  | ..\rockdev\userdata.img      |     |

Loader:

Run

Switch

Dev Partition

Clear

☐ Write by Address

Found One MASKROM Device

1-1 :MASKROM

注：烧写前，需安装最新 USB 驱动，驱动详见：

```
<SDK>/tools/windows/DriverAssitant_v5.13.zip
```

## 9.2 Linux 刷机说明

Linux 下的烧写工具位于 `tools/linux` 目录下(Linux\_Upgrade\_Tool 工具版本需要 V2.44 或以上)，请确认你的板子连接到 MASKROM/loader rockusb。比如编译生成的固件在 `rockdev` 目录下，升级命令如下：

```
sudo ./upgrade_tool ul rockdev/MiniLoaderAll.bin -noreset
sudo ./upgrade_tool di -p rockdev/parameter.txt
sudo ./upgrade_tool di -u rockdev/uboot.img
sudo ./upgrade_tool di -misc rockdev/misc.img
sudo ./upgrade_tool di -b rockdev/boot.img
sudo ./upgrade_tool di -recovery rockdev/recovery.img
sudo ./upgrade_tool di -oem rockdev/oem.img
sudo ./upgrade_tool di -rootfs rockdev/rootfs.img
sudo ./upgrade_tool di -userdata rockdev/userdata.img
sudo ./upgrade_tool rd
```

或升级打包后的完整固件：

```
sudo ./upgrade_tool uf rockdev/update.img
```

或在根目录，机器在 MASKROM 状态运行如下升级：

```
./rkflash.sh
```

### 9.3 系统分区说明

默认分区说明 ( 下面是 RV1126B EVB1 分区参考)

| Number | Start  | End    | Size   | Name     |
|--------|--------|--------|--------|----------|
| 1      | 8389kB | 12.6MB | 4194kB | uboot    |
| 2      | 12.6MB | 16.8MB | 4194kB | misc     |
| 3      | 16.8MB | 83.9MB | 67.1MB | boot     |
| 4      | 83.9MB | 218MB  | 134MB  | recovery |
| 5      | 218MB  | 252MB  | 33.6MB | bakcup   |
| 6      | 252MB  | 6694MB | 6442MB | rootfs   |
| 7      | 6694MB | 6828MB | 134MB  | oem      |
| 8      | 6828MB | 31.3GB | 24.4GB | userdata |

- uboot 分区：供 uboot 编译出来的 uboot.img。
- misc 分区：供 misc.img，给 recovery 使用。
- boot 分区：供 kernel 编译出来的 boot.img。
- recovery 分区：供 recovery 编译出的 recovery.img。
- backup 分区：预留，暂时没有用。
- rootfs 分区：供 buildroot、debian 或 yocto 编出来的 rootfs.img。
- oem 分区：给厂家使用，存放厂家的 APP 或数据。挂载在 /oem 目录。



- userdata 分区：供 APP 临时生成文件或给最终用户使用，挂载在 /userdata 目录下。

## 10. 常见问题解答（FAQ）

### 10.1 如何开启 DDR 变频

EVB 默认未开启 DDR 变频功能。如有开启需求，以 EVB1 为例，可通过以下配置进行开启：

```
--- a/arch/arm64/boot/dts/rockchip/rv1126b-evb.dtsi
+++ b/arch/arm64/boot/dts/rockchip/rv1126b-evb.dtsi
@@ -120,6 +120,15 @@ vcc12v_dcin: vcc12v-dcin {
    };
};

+&dfi {
+    status = "okay";
+};
+
+&dmc {
+    center-supply = <&vdd_logic>;
+    status = "okay";
+};
```

defconfig配置：

```
+ CONFIG_ARM_ROCKCHIP_DMC_DEVFREQ=y
+ CONFIG_PM_DEVFREQ_EVENT=y
```

```
# 查看DDR频率的可设置范围
cat /sys/class/devfreq/dmc/available_frequencies
# 设置DDR governors为userspace模式
echo userspace > /sys/class/devfreq/dmc/governor
# 设置DDR频率
echo 780000000 > /sys/class/devfreq/dmc/userspace/set_freq
# 查看当前DDR频率
cat /sys/class/devfreq/dmc/cur_freq
```

### 10.2 RV1126BP 无法读写的问题

在 Linux 系统中，若要确定文件系统的类型及是否可读写，可在 Kernel cmdline 中指定相关参数。例如，当系统为 ext4 格式且可读写时，可在以下位置进行如下设置：

```
--- a/arch/arm64/boot/dts/rockchip/rv1126bp-evb.dtsi
+++ b/arch/arm64/boot/dts/rockchip/rv1126bp-evb.dtsi
@@ -8,7 +8,7 @@

/ {
    chosen {
-       bootargs = "earlycon=uart8250,mmio32,0x20810000 console=ttyFIQ0
root=PARTUUID=614e0000-0000 rootwait snd_soc_core.prealloc_buffer_size_kbytes=16
coherent_pool=32K";
+       bootargs = "earlycon=uart8250,mmio32,0x20810000 console=ttyFIQ0 rw
root=PARTUUID=614e0000-0000 rootfstype=ext4 rootwait
snd_soc_core.prealloc_buffer_size_kbytes=16 coherent_pool=32K";
    }
}
```

## 10.3 如何读取CPU序列号

CPU序列号（即CPU ID）。

```
cat /sys/module/rockchip_cpuid/parameters/id
```